TP N°2
Amélioration du scénario du jeton

1. Affichage des Logs

Lors de la simulation, il est possible d'afficher des informations dans la partie log avec la primitive EV. Par exemple, le nœud affiche un texte lors de la réception du message.

Pour faire ceci , il faut rajouter le code source suivant dans la méthode HandleMessage() :

```
EV << "Le message " << msg->getName() << "est bien recu! Je le transfere a mon tour \n";
```

2. Ajout des variables d'état

Lors de la simulation, il est possible aussi de surveiller l'état des variables en temps réel. Par exemple, on veut surveiller l'état de la variable compteur utilisée pour controler le nombre de tours du message dans le scénario du jeton.

Pour faire ceci, il faut rajouter le code source suivant dans la méthode initialize():

```
WATCH (compteur) ;
```

Pour voir l'état de cette variable lors de la simulation, lancez la simulation et double cliquez sur l'un des nœuds. Une fenetre contenant les variables d'état observées pour ce nœud est affichée.

3. Ajout des paramètres

Dans le scénario du jeton, le nombre de tours que le message jeton doit faire avant qu'il ne soit stoppé est défini statiquement dans le code c++ (Nœud.cc). Il serait intéréssant de choisir ce nombre dynamiquement lors du lancement de la simulation. Ceci est le rôle des paramètres de simulation.

Il faut d'abord déclarer le nombre nb comme paramètre dans le module Nœud dans le fichier NED

```
simple Noeud
{   parameters:
    int nb;
    gates:
        input in;
        output out;
}
```

Ensuite, il faut récuperer, ce paramètre avec la primitive par dans la méthode Initialize() du fichier Nœud.cc :

```
nbr=par("nb");
```

- Nbr est la variable qui va reçevoir le nombre des tours affectée lors du lancement de la simulation.

A présent, lors du lancement de la simulation, une fenetre apparait pour vous demander d'indiquer la valeur du paramètre nb comme montré sur le shéma 2.

TP N°2
Amélioration du scénario du jeton

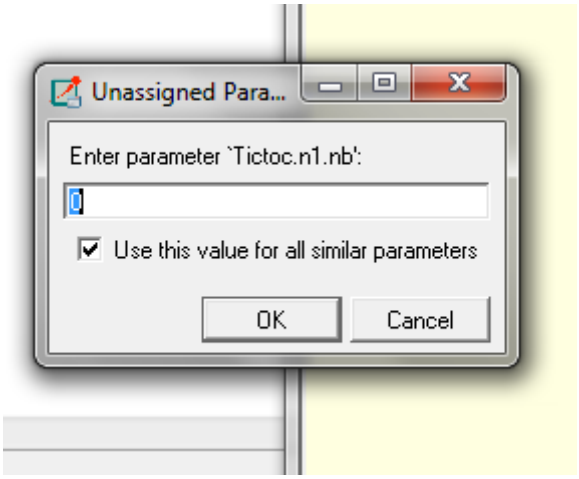


Schéma 1 les paramètres de simulation

Nous pouvons aussi spécifier ce paramètre dans le fichier d'initialisation (omnetpp.ini) :

```
Tictoc.*.nb=5
```

- Tictoc représente le nom du réseau tel qu'il est défini dans le fichier NED
- Le symbole * représente tous les nœuds du réseau Tictoc
- nb représente le paramètre du nœud.

4. Les Self-messages ou Timers

Dans les scénarios précédents, nous avons vu qu'un nœud fait un traitement lors de l'initialisation et lors de la réception d'un message. Ce ci est limité car le nœud dans ce cas la doit toujours dépendre des messages reçus des autres nœuds pour faire certains traitements or, il existe certains scénarios ou le nœud peut être seul dans le réseau par exemple, ou bien, le nœud doit faire un traitement périodique.

La solution d'est d'utiliser les self-messages (ou Timers) qui sont des messages programmés au niveau du nœud et envoyés à lui-même pour l'alerter qu'il faut faire le traitement en question.

Pour créer et programmer le self-message :

```
cMessage timer=new cMessage(" Timer")

scheduleAt(simTime()+5.0, timer);
```

- La fonction simTime() donne le temps de simulation écoulé.
- La primitive scheduleAt va programmer le message timer pour qu'il soit reçu par le nœud lui-même après 5 secondes.

TP N°2
Amélioration du scénario du jeton

Au niveau de la méthode handleMessage(), le nœud peut à présent recevoir deux types de messages : un message ordinaire envoyé d'un autre nœud et un self message envoyé de lui-même.

Pour savoir si le message reçu est un selfMessage, la méthode suivante est utilisée :

```
Msg-> isSelfMessage()
```

Elle retourne vrai si le message est un self message, faux sinon.

5. Exercice d'application

Pour le scénario suivant, nous allons garder la topologie en anneau du TP précédant mais avec 10 nœuds (n1, n2 ……, n10). Le nœud n1 envoi périodiquement un message au nœud suivant et ainsi de suite chaque nœud transfère ce message jusqu'à ce que le message arrive chez le nœud n10. Ce dernier va alors le supprimer avec la primitive delete(msg). L'envoi périodique se fait **nb** fois tel que nb est défini dans le fichier omnetpp.ini. Le délai périodique **d** de l'envoi du message est également spécifié dans le fichier omnetpp.ini.

Pour le test du scénario : **nb**=8 et **d**=5